

Sprawozdanie z pracy projektowej

System biblioteczno-sklepowy *meow*

ASP.NET Core MVC + REST API

Zespół projektowy PPU

Repozytorium: <https://github.com/04Sqv3r/s4-ppi1>

12 czerwca 2026

Spis treści

1	Opis przedmiotu zlecenia	2
1.1	Zakres funkcjonalny	2
2	Opis technologiczny proponowanego rozwiązania	2
2.1	Architektura ogólna	2
2.2	Stos technologiczny	3
2.3	Model danych	3
2.4	Warstwa API i cache	3
2.5	Uwierzytelnianie i autoryzacja	4
2.6	Lokalizacja i interfejs	4
2.7	Integracje zewnętrzne (stan implementacji)	4
3	Wykaz zadań projektowych z szacowanym czasem	4
4	Instrukcja uruchomienia systemu	5
4.1	Wymagania wstępne	5
4.2	Uruchomienie lokalne	5
4.3	Uruchomienie zdalne (produkcja)	6
4.3.1	Docker Compose (VPS)	6
4.3.2	Railway (zalecane dla demo)	6
4.4	Pipeline CI	6
5	Dokumentacja OpenAPI	6
5.1	Dostęp do dokumentacji	7
5.2	Endpointy	7
5.3	Przykład wywołania	7
6	Podsumowanie	7

1 Opis przedmiotu zlecenia

Przedmiotem zlecenia projektowego jest zaprojektowanie, implementacja i wdrożenie zintegrowanego systemu informatycznego łączącego **wypożyczalnię książek** oraz **sklep internetowy** pod wspólną marką *meow*.

Zgodnie z wymaganiami prowadzącego przedmiot *Programowanie urządzeń mobilnych* system musi udostępniać:

- **interfejs webowy** dla klientów (przeglądanie katalogu, koszyk, checkout, rezerwacja egzemplarzy bibliotecznych, konto użytkownika),
- **panel administracyjny** do zarządzania katalogiem, klientami, wypożyczeniami i zamówieniami sklepowymi,
- **REST API** (z dokumentacją OpenAPI) jako warstwę danych dla planowanej aplikacji mobilnej.

1.1 Zakres funkcjonalny

System obejmuje m.in.:

- rejestrację i logowanie użytkowników (role: klient, administrator),
- zarządzanie książkami, egzemplarzami bibliotecznymi i stanem magazynowym sklepu,
- proces zakupu (koszyk, dane dostawy, składanie zamówienia),
- wypożyczenia, rezerwacje, zwroty oraz obsługę opóźnień,
- eksport danych raportowych z modułu wypożyczeń,
- integrację z zewnętrznymi usługami (cache Redis, SMTP, szkielet AI/Ollama).

2 Opis technologiczny proponowanego rozwiązania

2.1 Architektura ogólna

Aplikacja oparta jest na wzorcu **MVC** (Model–View–Controller) w środowisku **ASP.NET Core** (.NET 10). Warstwa prezentacji realizowana jest przez widoki Razor (.cshtml); logika biznesowa znajduje się w kontrolerach i serwisach; dane utrwalane są w relacyjnej bazie **MySQL 8** przez **Entity Framework Core** (Pomelo provider).

Klient WWW → **ASP.NET Core (MVC + API)** → **MySQL**
↔ **Redis** (cache katalogu) ↔ **MailKit/SMTP** (powiadomienia) ↔
Qdrant (wektory, opcjonalnie)

Rysunek 1: Uproszczony schemat architektury systemu *meow*.

2.2 Stos technologiczny

Tabela 1: Technologie wykorzystane w projekcie.

Warstwa	Technologia
Backend	ASP.NET Core MVC, C#
Baza danych	MySQL 8.0, EF Core (Code First)
Cache	Redis 7 (StackExchange.Redis)
API	REST, JSON, Swagger/OpenAPI (Swashbuckle)
Auth (WWW)	Sesje ASP.NET Core, BCrypt
Auth (API)	JWT Bearer
Frontend	Razor Views, Bootstrap 5, CSS/JS
Lokalizacja	IStringLocalizer, pliki .resx (PL/EN)
Mailing	MailKit
Konteneryzacja	Docker, Docker Compose
CI	GitHub Actions (<code>dotnet build</code>)

2.3 Model danych

Główne encje ORM (`LibraryDbContext`):

- `Book` — tytuł, autor, gatunek, ceny, ilość w sklepie,
- `Egzemplarz` — egzemplarz biblioteczny (numer inwentarzowy, stan),
- `Klient`, `User` — dane klienta i konto logowania,
- `Wypozyczenie`, `Platnosc` — wypożyczenia i kary,
- `Zamowienie` — zamówienia sklepowe.

Migracje EF Core stosowane są przy starcie aplikacji (`Database.Migrate()`).

2.4 Warstwa API i cache

Publiczne endpointy REST (prefix `/api`):

- `GET /api/books` — katalog z opcjonalnym filtrem `?q=` i `?gatunek=`,
- `GET /api/books/{id}` — szczegóły książki,
- `POST /api/auth/login` — uwierzytelnienie JWT,
- `POST /api/books/refresh-cache` — unieważnienie cache (dev/admin).

Serwis `BookCatalogCacheService` buforuje wyniki katalogu w Redisie (klucze z prefiksem `meow:`), co ogranicza obciążenie bazy przy częstych odczytach (np. live search w sklepie).

2.5 Uwierzytelnianie i autoryzacja

- **Panel WWW:** sesja po logowaniu (`HttpContext.Session`), atrybut [`MeowAuthorize(Rola = "Admin")`] na akcjach administracyjnych.
- **API mobilne:** token JWT (HS256, issuer/audience w `appsettings.json`), ważność 8 h, claim `klient_id` dla powiązania z profilem.

Hasła przechowywane są jako skróty BCrypt.

2.6 Lokalizacja i interfejs

Interfejs obsługuje języki **polski** (domyślny) i **angielski**. Przełącznik kultury zapisuje wybór w ciasteczku (`CookieRequestCultureProvider`). Teksty UI w plikach `Resources/SharedResources`. Responsywność (RWD) zapewnia `wwwroot/css/responsive.css` oraz komponenty Bootstrap 5.

2.7 Integracje zewnętrzne (stan implementacji)

- **Redis, Swagger, MailKit** — zaimplementowane; SMTP w trybie dev loguje treść maila do konsoli (`[MAIL-DEV]`).
- **Ollama (AI)** — szkielet (`AITestController`); wymaga osobnej instalacji Ollama poza `docker-compose.yml` (w trakcie integracji).
- **Qdrant + ONNX** — infrastruktura Docker i `VectorSearchService`; wykorzystanie produkcyjne planowane.

3 Wykaz zadań projektowych z szacowanym czasem

Poniższa tabela przedstawia główne zadania realizacji systemu wraz z orientacyjnym czasem potrzebnym do wykonania (bez przypisania do konkretnych osób).

Zadanie	Czas	Rezultat
Analiza wymagań i projekt architektury	8–12 h	Diagram encji, podział MVC/API
Konfiguracja repozytorium Git, Docker, MySQL	6–10 h	<code>docker-compose.yml</code> , migracje EF
Moduł sklepu (katalog, koszyk, checkout)	20–30 h	<code>ShopController</code> , widoki sklepu
Moduł biblioteki (wypożyczenia, zwroty, rezerwacje)	20–28 h	<code>RentalsController</code> , panel zwrotów
Panel administracyjny (książki, klienci, dashboard)	16–24 h	<code>BooksController</code> , <code>ClientsController</code>
REST API + Swagger/OpenAPI	10–14 h	<code>Controllers/Api/</code> , <code>/swagger</code>
Cache Redis	4–6 h	<code>BookCatalogCacheService</code>

Zadanie	Czas	Rezultat
Uwierzytelnianie (sesje + JWT)	8–12 h	AccountController, AuthApiController
Lokalizacja PL/EN	12–18 h	SharedResources.*.resx
Mailing (MailKit)	4–6 h	EmailService
Interakcje asynchroniczne (live search)	3–5 h	live-search.js
RWD i Bootstrap 5	6–10 h	responsive.css, layout
Integracja AI (Ollama)	10–16 h	AITestController (w trakcie)
CI/CD i dokumentacja wdrożenia	4–8 h	.github/workflows/ci.yml, DEPLOY.md
Hosting produkcyjny (Railway/VPS)	4–8 h	Publiczny URL aplikacji
Testy, poprawki, sprawozdanie	10–16 h	Stabilna wersja demo
Łącznie (szacunek)	145–223 h	

4 Instrukcja uruchomienia systemu

4.1 Wymagania wstępne

- Docker Desktop (Windows/macOS) lub Docker Engine + Compose (Linux),
- Porty wolne: 5000 (WWW), 3307 (MySQL), 6379 (Redis), opcjonalnie 6333 (Qdrant),
- Klon repozytorium:

```
git clone https://github.com/04Sqvr3r/s4-ppsi1.git
cd s4-ppsi1
git checkout filip/aktualna-wersja # lub main po merge
```

4.2 Uruchomienie lokalne

W katalogu projektu:

```
docker compose up --build
```

Po zbudowaniu kontenerów:

- Strona WWW: <http://localhost:5000>
- Dokumentacja OpenAPI (Swagger UI): <http://localhost:5000/swagger>
- Specyfikacja JSON: <http://localhost:5000/swagger/v1/swagger.json>

Pierwsze konto: zarejestruj użytkownika przez /Account/Register, następnie zaloguj się. Rolę administratora nadaje się w bazie:

```
UPDATE Users SET Rola = 'Admin' WHERE Login = 'twoj_login';
```

Zatrzymanie:

```
docker compose down
```

4.3 Uruchomienie zdalne (produkcja)

4.3.1 Docker Compose (VPS)

```
docker compose -f docker-compose.yml \
-f docker-compose.prod.yml up -d --build
```

Wymagane zmienne środowiskowe:

- `ConnectionStrings__DefaultConnection`
- `ConnectionStrings__Redis`
- `Jwt__Key` (min. 32 znaki)
- opcjonalnie: `Smtp__Host`, `Smtp__User`, `Smtp__Password`

4.3.2 Railway (zalecane dla demo)

1. Utwórz projekt na <https://railway.app> i połącz z repozytorium GitHub.
2. Dodaj usługi **MySQL** oraz **Redis**.
3. W aplikacji webowej ustaw m.in.:
 - `ASPNETCORE_URLS=http://0.0.0.0:8080`
 - `ConnectionStrings__DefaultConnection` — z panelu MySQL
 - `ConnectionStrings__Redis` — z panelu Redis
4. Railway wykrywa Dockerfile i buduje obraz automatycznie.
5. W zakładce *Networking* wygeneruj publiczny URL.
6. Dokumentacja OpenAPI dostępna pod adresem `https://twoja-domena/swagger`.

Szczegóły: plik `DEPLOY.md` w repozytorium.

4.4 Pipeline CI

Przy pushu na gałąź `main` uruchamiany jest workflow GitHub Actions (`.github/workflows/ci.yml`): `dotnet restore`, `dotnet build`, `dotnet publish`.

5 Dokumentacja OpenAPI

Specyfikacja API generowana jest automatycznie przez `Swashbuckle.AspNetCore` i udostępniana w formacie OpenAPI 3.0.

5.1 Dostęp do dokumentacji

Tabela 3: Adresy dokumentacji API.

Zasób	URL (lokalnie)
Swagger UI	http://localhost:5000/swagger
Plik OpenAPI (JSON)	http://localhost:5000/swagger/v1/swagger.json

Po wdrożeniu produkcyjnym te same ścieżki działają pod publicznym adresem hosta (wymaganie: dokumentacja OpenAPI dołączona do hostowanej aplikacji).

5.2 Endpointy

Tabela 4: Publiczne endpointy REST API (wersja v1).

Metoda	Ścieżka	Opis
GET	/api/books	Lista książek. Parametry: q (fraz), gatunek . Cache Redis.
GET	/api/books/{id}	Szczegóły książki po identyfikatorze.
POST	/api/auth/login	Logowanie; body JSON: { "login", "haslo" }; odpowiedź: token JWT.
POST	/api/books/refresh-cache	Czyści cache katalogu (204 No Content).

5.3 Przykład wywołania

Logowanie:

```
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"login":"user","haslo":"haslo123"}'
```

Katalog z wyszukiwaniem:

```
curl "http://localhost:5000/api/books?q=wiedzmin"
```

Autoryzacja JWT (nagłówek):

```
Authorization: Bearer <token>
```

6 Podsumowanie

System *meow* realizuje zintegrowany model biblioteki i sklepu internetowego z panelem administracyjnym oraz warstwą REST API gotową pod aplikację mobilną. Rozwiązanie konteneryzowane (Docker) umożliwia jednolite uruchomienie lokalne i zdalne; dokumentacja OpenAPI dostępna jest przez Swagger UI pod ścieżką **/swagger**.

Elementy w trakcie dokończenia: pełna integracja Ollama, hosting produkcyjny z publicznym URL oraz domknięcie tłumaczeń wybranych formularzy administracyjnych.